



Testing Apollo 3

Course Overview

Course Overview

Hi, everyone. My name is Marques Woodson, and welcome to my course, Testing Apollo. I am a staff software engineer at Project44. The GraphQL technology has been taking off over the past couple of years, and the Apollo framework is one of the most popular tools to use with GraphQL. This course is designed to teach you how to test your full stack Apollo application. Some of the major topics that we will cover include testing queries and mutations in your React application, including how to mock calls with MOTS provider, unit testing your resolvers on the server side with Apollo Server, integration and snapshot tests with Apollo Server, and using introspection to get the schema definition of a GraphQL database. By the end of this course, you'll know how to properly test your Apollo client and your Apollo Server application. Before beginning this course, you should be familiar with the basics of Apollo and GraphQL. I hope you'll join me on this journey to learn how to test an Apollo application with the Testing Apollo course at Pluralsight.

Apollo Testing Introduction

Course Introduction

Hi, I'm Marques Woodson, and welcome to my Pluralsight course Testing Apollo. I'm a software engineer with over 16 years of experience with my primary focus on JavaScript. Over the years, I've been able to work with many different frameworks, including React, Angular, Vue, and Next.js to name a few. My work was usually split between the client side and the server, which was running on Node.js. The work I've done on the server involved tools like Express, MongoDB, and GraphQL. That brings us to the topic of this course, Apollo. Apollo is a GraphQL framework used to develop full-stack web applications. In this course, I'll show you how to test a full-stack Apollo application. On the client side, we'll be using the Apollo Client library with the React application. And on the server, we'll use Apollo Server. You'll learn how to unit test React components that use Apollo to fetch and mutate data from an Apollo server. I'll go over how to test the loading, final, and error states when making queries from a component. One of my favorite tools with writing tests in React is the React Testing Library or RTL. React Testing Library was created by Ken C. Dodds in order to provide best practices and standards with how we write tests. It enables us to find and select elements



in the DOM as if a user who was looking for them. For example, a user might look for a button with the text submit, not for a button with a class or ID of submit button. The RTL package has a ton of helper methods to make writing tests easy, and we'll be using RTL throughout this course. You'll also learn how to test queries and mutations in your resolvers inside Apollo server. You'll learn how to use MockLists in your resolvers to more easily mock your data and schema mocking using introspection. I'll also show you how to run integration tests, which involves testing the Apollo Server without having to make real HTTP requests. To get the most out of this course, it's important that you have an understanding of how hooks work in React since we'll be using hooks to manage state and side effects in the components. You should also have some experience with using Apollo to create a small application. This course focuses on testing an Apollo application, and I won't go too far into the details of how to write an Apollo app. I'll definitely give an overview of many Apollo features that aren't particularly testing-related, but they will be short and concise. You should also have some experience with writing code in a Node.js environment since that's where all of the Apollo Server functionality will live. You don't have to be a Node expert, but some experience will be helpful.

Apollo Client Features

Let's talk a little about the Apollo Client. Apollo Client is a feature-packed library used to fetch data from a GraphQL back end in a predictable, declarative way. React is officially supported by the Apollo team, but there are also implementations in Vue, Svelte, Angular, and Ember that are community supported. Speaking of React, what we'll be really interested in are the hooks provided by the Apollo Client, `useQuery` and `useMutation`. There are a couple of methods for data fetching on the Apollo Client. One is to use a `useQuery` hook. If you aren't very familiar with React hooks yet, don't worry. The most important thing to remember is that hooks only work in functional components. They won't work in a class component. If for whatever reason you don't want to use the `useQuery` hook, there is another option, the `query` component. This component uses the `render prop` pattern where the `children` prop is a function with parameters of `loading`, `data`, and `error`. You would work with the return data inside of the `query` component. In this course's demo application, I will be using the hooks that Apollo Client provides instead of the components. `useQuery` is a custom hook that utilizes the native React hook `useEffect` among other hooks. The `useEffect` hook is meant for updating your component whenever specified values in the component change. So `useEffect` callbacks can run only once when a component mounts, or it can run every time a component rerenders, or it can run only when a specific value changes. It's up to you. Usually you use the `useEffect` hook to work with side effects or asynchronous code. The side effect that `useQuery` initiates is making the API call to a GraphQL server to fetch or mutate data. The `useQuery` custom hook is taking care of all of the



details behind the scenes, so you only need to worry about constructing the query. `useQuery` returns three values, loading, data, and error. The loading property is a boolean that lets you know that work is still happening. You would use this property to show a loading spinner or some other way to give feedback to your users that things are happening and to please wait. If the query throws an error, loading would be set to false, and the error property will be defined with an error object. This property will look different based on the error policy used with the request. Every request can have a different error policy applied. By default, the error policy is set to none, which means every GraphQL error is handled as if it were a network error. You also have the option to ignore the GraphQL errors by setting the error policy to ignore. Doing so would mean both the data and error properties would return as undefined. I would use the ignore policy sparingly since it provides the least amount of information on what's happening with the request. The last available error policy is the all policy. This is the recommended policy as it will allow you to handle any errors in the UI, as well as still display any successful data that was returned. So if one field fails, you are still able to display the other fields. I'll show a real example of this later on. In the course. The data property would be defined when there are no errors and loading has finished. It holds the data that is returned based on your query. There's a lot you can do with the `useQuery` hook, including customizing caching, refetching, and polling for new results. And in this course, I'll cover testing what I think are the common use cases. Not only will we fetch data from our Apollo Server, we will also want to edit data. As you'll see later in this demo application, we will need to have the ability to update or mutate some data that we'll pull from the server. We'll use another hook from the Apollo Client library to achieve that, the `useMutation` hook. The `useMutation` hook takes a mutation query as the first argument and returns a tuple with an update function at the first position and an object with loading and error states in the second position. The loading and error states object works just like the `useQuery` loading and error states. The update function is needed to actually execute the mutation. Just calling `useMutation` itself will not execute the mutation query. You will most likely call the update function when a button is clicked or a form is submitted or some other type of interaction happens. If your query requires different variables, like the ID, you are able to pass these to the update function to make sure you are updating the correct data. Next, we'll go over the mock provider and how it's used for testing React components that include Apollo.

Mocked Provider

React components that are utilizing Apollo Client tools for querying and mutating GraphQL data rely on a native React feature called context. Context is a way for components to share data and functionality with each other. The Apollo Client library gives us a context provider to wrap around our application so that we can use the Apollo hooks or Apollo components within our own component tree. The



Apollo provider requires an Apollo Client as a property. This is how we connect to an Apollo Server instance, and also where we configure things like cache, server-side rendering, and default query options. Context providers are becoming more and more common in React applications today. If you're using Redux or internationalization tools like React Intl, for example, there's a good chance you are wrapping your main application component tree with provider components. For example, React Intl requires you to wrap your component tree with the IntlProvider component so that you can use multiple languages throughout your component tree. Once you start writing tests for the child components of these providers, you'll realize that sometimes you need to include these providers inside of your tests. Here I have a sample app with a component that uses three useQuery hook. To give this component access to Apollo, I need to make sure it wraps with the Apollo provider. Now if I were to try to render it this component in a test, you'll receive an error right away that says cannot find client in the context or pass as an option. Wrap the root component in an ApolloProvider. This error is very helpful since it tells you exactly how to fix your test. You might think, I'll just export the same client I'm using in index.js and use it in my tests. The issue with that is the real client makes actual HTTP requests. Your unit tests shouldn't interact directly with a back end. This is where Apollo's MockedProvider component comes in. MockedProvider will mock calls to your GraphQL endpoint and give you the ability to specify the results of each of your requests. This works by passing the MockedProvider an array of request response objects. A request object has a query property where its value is the query that you want to watch for. It also has a variables property that you can use to specify the value of any variables that are used in the query. To go along with each request, you need to specify the results. Successful results should have a data property with its value being some mock data for your test. This request results object is passed to the mocks prop on the MockedProvider. MockedProvider also has an addTypeName property that we'll set as false. This is important to make sure the mocks query matches what is run in your actual component. When your component runs a query, Apollo Client will add the typename property to that query automatically. This helps Apollo cache your query results under the hood. In this test example, we're passing in the raw query, which will not have the typename prop. We set addTypeName as false so that the queries will match correctly. If you don't plan to pass in any request response objects to the mocks property, like in the case of testing a loading state, you can exclude their typename property as well. Later in this course, we'll be using the MockedProvider when we're writing tests on the client. I'll cover how it works when doing both queries and mutations.

Apollo Server Features

Seeing as this is a course about testing Apollo, we can't leave out Apollo Server. All of the queries and mutations that we'll be executing in the React App



will be interacting with an Apollo Server that we'll be running in a code sandbox container. Topics that we want to test on the server include resolver functions, including both queries and mutations, schema mocking with introspection, integration tests. Those are the most valuable. I do want to take a little bit of time to give a quick overview of the basics of an Apollo Server. I won't go too deep into the weeds, but I want to make sure that these topics are fresh in your mind before we get to writing tests. There are two main components that you need with your Apollo Server, a schema and resolvers. A schema is used to define the structure of your data. It establishes the different types of your data and the relationship between those types. For example, if you plan to have a type of user in your graph, that user itself is a type, and all of the properties that belong to that user will have types. Like the user's name would be of type string, the user's age of type number, and so on. It's very similar to TypeScript or Elm. Two special types will be important to know, the query type and the mutation type. The query type defines entry points to all of your data in the graph. For example, take a look at this query. I'm attempting to query the graph for a list of products, and I want to return the id, name, and price of each product. For this to work, we need to have a query type with a products field defined in our schema. It's how we tell the Apollo Server what is queryable and what is not. Here I'll create a basic schema that defines a Query type with a products field. I'll need to define the Product type now, just to keep things clean. I'll just add a name field of type string, and id field as type id. We'll talk about the id type later. And a price field also of type String. So what we have here now is the Query type with a single field, products. Products will return an array of objects of type Product. That's the purpose of the square brackets around the product type. Now to have a working client-server-Apollo relationship, there's still more to do. However, this is a necessary addition to the schema to be able to query products. When we want to edit our products, we must take a similar approach with mutations. Mutations are the second special type that we can utilize in our schema definition. Think of mutations as write operations, while queries are the read operations. As an example, we can define a mutation for adding a new product. I'd make the type Mutation, then add addProduct, which will be a function that accepts two arguments, a name and a price. This mutation will return a product as a response. Return types of functions look the same as when we define a return type in TypeScript. In most cases for mutation types, the property defined is a function, not just a field. That's because we are normally passing arguments to a mutation. Since our product definition has a name of type String and a price of type String, those are reflected in the arguments of this mutation. Now we have our Apollo schema defined with both a query type and a Mutation type. We've also defined the product type that is reusable and used in multiple places so far. The next requirement for having a working Apollo Server instance is setting up the resolvers. A resolver is responsible for populating data for every field you have defined in your schema. Each resolver resides in a JavaScript object called a resolver map. The map consists of top-level objects which have keys that match with the types defined in your schema, like



query, mutation, or products, where the descendants of those types are functions. The names of those functions match with the field names that you have defined in your schema. Something to be aware of is you aren't required to manually define a resolver for every single field in your schema. Apollo Server will use default resolvers for whatever you don't define yourself. The default resolver will check to see if the parent object has a property with the exact name of the field that was queried. If it does, it'll return the property's value. When I say parent object, I mean the top-level object in the resolvers map. For example, let's define a product resolver function for the price field. The top-level key will be product to match with the product type in the schema. Inside of the product object I'll define the price function. The first parameter is the parent object, and there are three more params that aren't important right now, so I'm going to ignore those. Now, the parent object is the product object, so it will have an id and a price available. We can just return `parent.price` in this resolver function, if we wanted to. Another thing we could do is apply some business logic to the price before returning it to the client, like converting dollars to euros or anything, really. I'm not going to create a resolver for the name field because I don't need to do anything extra that the default resolver doesn't already do. The default resolver will either return `parent.name`, if it is defined in the data, or it will return null. Later on in this course we'll be writing tests for our resolvers that revolve around querying data, but we also will need tests around mutation resolvers. Writing a mutation resolver works just like writing resolvers for queries. However, the top-level key will be mutation. Based on our schema, we do have an `addProduct` method defined inside of the mutation type. Defining the resolver for this method is simple. I'll add the `addProduct` function inside of the mutation resolver map, and this function still takes four parameters, just like what we wrote in the price resolver. The main difference here is that we are going to use one of the other parameters available. The second parameter in the resolver function is the arguments object. This object holds any arguments passed to the `addProduct` function. And as you can see, we define two arguments in the schema, a name and a price. So for the `args` parameter, we'll have a name and a price to use while creating a new product. In a real application, this resolver would insert the new product into a database or something similar. And actually, the demo app that I'll give you an overview of later on makes use of both queries and mutations, so you'll definitely get some practice testing these common types.

Apollo Mocks on the Server

Before we go over the demo application and begin writing tests, I wanted to explain how we can use mocks in an Apollo Server instance. Honestly, Apollo makes it incredibly simple to get started with mocking your data with just one line of code. When you create a new Apollo Server instance along with passing your type definitions and resolvers, add the property `mocks` and set that as `true`. This is all you need for mocking your data. Setting `mocks` to `true` tells the Apollo



Server to use your schema to generate mock data based on the types. So if a field has a type of string, a string would be returned. If a field was a number type, a random number would be returned, and so on. These basic default mocks definitely get us to a good starting point, but there are some drawbacks. First, any resolvers that are supplied will be ignored. This is not ideal because, most likely, you want to be able to still use your resolver logic in your tests. Also, just setting mocks as true means you are able to customize the return values. Yes, a string would return a random string, but maybe you need those random strings to be a certain length or limited to a set of strings like North American states only. To customize your mocks, all you need to do is instead of passing the Boolean true to the mocks property, pass an object where each key represents the type that you would like to mock, and the value is a function that returns your custom values. For example, you can set Int as a key and for all ints return the value 1000. This means that anywhere in your schema that you define a type of int, that value will return 1000. You could do the same with the string type. If I define a string key on the mocks object, I can have the function return any string I want. We can use the same amount of control mocking resolvers as well. When you want to mock a query resolver, you would set the key to query in the mocks object. The value is a function that returns an object that contains your resolver fields, just like we have in the resolver map. Take our products resolver, for example. If I wanted to mock the products resolver, I would have the query function return an object that holds the key products, and the value of products will be a function that returns some mock data. Now products is meant to be a list of different products, so I could manually define an array of products here. This could get tedious and difficult to maintain quickly. Apollo gives us a better solution, MockLists. MockList is a function that allows you to generate a list of data based on your schema definition. You specify exactly how many items you want in the list, or you can declare a range of items, so you can say I want anywhere between 5 and 15 items to be returned. The best part is the products that get returned will have the shape and structure that was defined in the schema. Apollo Server is smart enough to generate the correct data for us. By default, when you either set the mocks property to true or an object with customized mocking, your entire set of resolvers will be mocked. Again, this is great for a lot of use cases, but sometimes you don't want to mock all resolvers. If you would like to keep your existing resolvers that you've defined, you can use a special property called mockEntireSchema in the ApolloServer config and set its value to false. Doing so means any existing resolvers that you've manually defined will not be mocked. The scenario may come up for some of you where your schemas were not written in JavaScript. Maybe your schema was defined in Python or something similar. Well, GraphQL does have a way for you to run a query that will return details about the schema regardless of the language it was written in. This feature is called introspection. Introspection isn't a feature only available with Apollo. It's a GraphQL feature. You run an introspection query similar to other queries except the query subject is `_schema`. The fields to return could be name to get the name of the type defined in the schema and kind to get



the kind of type or type kind. Introspection can give you all of the information you need to know about what the schema looks like. Now, if we would like to mock the results of your introspection queries, GraphQL provides a function for building the schema based on the introspection results. That function is called `buildClientSchema`, and it comes from the GraphQL package. Once you have the JSON returned from your introspection results, you can import those into your ApolloServer config and pass them to the `buildClientSchema` function as its only argument. `BuildClientSchema` returns a schema object, which you can then pass to your ApolloServer config just like if you wrote the schema yourself.

Unit Testing an Apollo Client

Demo App Introduction

It's finally time to go over what's included in our demo application. Earlier in this module, I've talked about many of the different Apollo Client features that we'll be testing in this demo application. I've also gone over some of Apollo Server's features, all of which are going to be used in the demo app. This application is called Carved Rock Fitness and is an e-commerce store focusing on the outdoors. There's information on hiking, canoeing, rock climbing, and all of the gear needed for these different activities. This application is written in React, and we'll be using Apollo Client version 3, the latest version available. If you are still using version 2, you will still give value out of this course. The biggest changes that came with version 3 are that the Apollo modules have been consolidated into a single package, `@apollo/client`. There are some breaking changes, however, all of which can be found in the Apollo Client change log. The part of the application we'll be testing will revolve around these stories. These stories are here to help show the visitor all of the possibilities of what Carved Rock's products have to offer. We don't want these to be static so we'll pull this data from a graph UL database using the Apollo Client. Later, we'll build out the components for the stories and use Apollo Client to fetch the data. We'll also want to use this UI to update a story. This will let us have a chance to perform a mutation and utilize the `useMutation` hook in our component. Then I'll show you how to write tests around mutations and queries. For the React app, you should be able to download the source code in the Exercise Files section on Pluralsight's UI and follow along with the course. The server side will be hosted in CodeSandbox. You should follow the CodeSandbox link and fork the container. Once we start getting into the client source code, I'll show you where you'll place the link to the server when configuring the Apollo Client.

Connect the Client to the Apollo Server



We are now at a point where we can start getting into the code of our demo app. First, we need to get the project set up by installing the dependencies. Then we'll make the stories component dynamic by pulling each story from our Apollo server. We're going to start with writing some tests for the React app, including tests for queries and mutations, and we'll be utilizing the `MockedProvider` component. We'll also make sure we test the error states of the `useQuery` and `useMutation` hooks. First thing we want to do is run `yarn install`, just to make sure we get all of our dependencies. Now we'll need to add in the `ApolloProvider` to give the Carved Rock Fitness application access to the Apollo Server. Head over to `index.js` and import the `ApolloProvider` component from the `@apollo/client` module. We actually need to also pull in a couple of other functions, the `ApolloClient` function, the `InMemoryCache`, and the `HttpLink`, all of which will be used to configure the `ApolloClient`. To create our client, we need to call `new ApolloClient`, and we're going to pass a configuration object that should consist of a cache, which will just use an instance of the `InMemoryCache`; and a link, which will be an `HttpLink` to the Apollo Server. Now the uri that you want to place here in the `HttpLink` is the same URL that you'll need in the codesandbox browser address bar. When you visit the codesandbox link that you got from the slides, you'll need to fork the sandbox, which will refresh the page and give you a new sub-domain for the running Apollo Server. You can see the URL here, and I'll just want to copy that and bring that over and paste it into the uri of the `HttpLink` config. Now we need to wrap our app component with the `ApolloProvider` and pass the client we just created as the only property. This is pretty much all we need for the index file, and now the application should have access to the different hooks and components in the `ApolloClient` library that we'll need for interacting with the Apollo Server. Over in the Stories component, we can see the code that makes up this section of the application here. Each one of these, the Splash Chic, the Kid Klimbers, these are each different stories. We want to change this component to instead of having static stories, we want to pull these from the Apollo Server. Let's take a second and look at our codesandbox for the Apollo Server, and you will notice a `stories.json` file. This is the data for each story that we want to display in the application. Each story has an id, a name, an image link, and a description. This data matches up with what we have in the static code in the React app right now. To have our server return this data, we need to make sure we have our schema defined with the stories query and a type for a story, and we want to update our resolvers to read from this file and return the results. Let's create a `Query` type, and the property we want to define is `stories`, which will return an array of the type `Story`, which we'll define next. And for the `Story` type, we will have the property `id`, which will have a type of `ID`, and it is required. That's what this exclamation point means. We also need the field name, which is a `String`, the `image`, which will also be a `String`, and finally, the `description`, which is also a `String`. That should be enough for our type right now. Next, we need to define a query resolver for the stories field. Remember that the resolver function has parameters that we aren't actually going to use right now, but I'm still going to put them here just so you can see what a resolver



function would look like. We'll use the fs library to read the stories.json file, and we can return the JSON parsed results. So let's require the fs library. Then use `readFileSync` to read the contents of stories.json. Then we can return `JSON.parse(results)`. To quickly check if this is actually working, just go over here to the browser and we can test out a stories query. You can do a quick stories query where you want to get back the name, image, and description. Click the Play button, and there you should see the stories.json data returned. Now, with the query ready to go, we can go back over to our React app and start querying for this data in the Stories component. We should first import the `useQuery` hook and the `gql` function from the Apollo library. `Gql` is going to be used to actually generate a GraphQL query. We'll go ahead and write that query now. Let's call this `STORIES_QUERY`, and we'll use `gql` and then a back tick, and we'll pretty much write the same query we just wrote in the Apollo Server. So stories, and then we'll want back id, name, image, and description. To execute this query, we'll use the `useQuery` hook inside the Stories component. So let's destructure the loading, error, and data properties from the response from calling `useQuery`. And we'll pass the `STORIES_QUERY` as the only argument. The loading property will be a Boolean, and if it's true, we want to return the message Loading. If error is populated, if it's not undefined, we want to return the message Error. If neither of those conditions are met, we can start interacting with the data property. The data property should have a stories object, which would be an array of those stories that we saw in Apollo Server. We should loop over those stories and build an array of markup that we'll put in the return of this component. So let me go ahead and copy the div that will hold the story information, create a variable called stories, and we will loop over `data.stories`. Paste in that div that we just copied, add a key on the parent div here. Set that value to `story.id`, since when you're creating an array like this we need to make sure the key is there and it is unique. Now we just need to replace the static data with the data coming from the database. So in the image we'll set the alt tag to be `story.name`, and set the source to be `story.image`. In the `h3`, we'll add `story.name`, and just below that we'll put `story.description`. Now we can delete all of the static information here, and we'll replace it with the stories variable. At this point, this component should be working, it should be pulling the information from the Apollo Server, so let's just check the browser and make sure we are looking good. Yes, the stories are loading, so we are successfully connected to the Apollo Server.

Testing Queries in React

It looks like it's time to run our first test. Go ahead and create a file called `stories.spec.js`. Now we need to import some dependencies starting with React. Next, we want to grab the render and screen methods from the testing library, and we will also need to grab the `extend-expect` module from testing-library to get a few helper expectation methods. And finally, let's import the stories component, since that is what we're going to be testing. First, we'll need a



describe block for the stories component. In the first test, we will check to see if the loading message is displayed while the component is fetching data. Go ahead and render the stories component, using the render method from the testing library. Then I'll want an expectation that the text loading will be in the document. We'll use this method `screen.getByText`, and we'll pass loading as a regular expression, and we'll use the assertion to be in the document. Technically, you do not have to use a regular expression, I just prefer to use these in my tests. Showing the test results, we'll already see our test failing, because we are not passing in a client or an Apollo provider is not wrapping our stories component, which we talked about earlier. At this point, we are going to want to pull in the mocked provider component to wrap our stories component so that we can actually have access to the use-query hook. `MockedProvider` comes from the `@apollo/client/testing` module. We need to create a mocks object to pass to the `MockedProvider`, which is just an object with a request and result pair for every request you want to use. In our case, we have one request, which is the stories query. We can actually import that stories query from the stories component and pass it as the value to the query property here under the request object. But first we need to export it from the stories component. So let's go back and just add an export. Now that we can import the stories query inside of our test, let's go ahead and fill out this box object. In the request object, we only need one field right now, and that's the query property where the value will be `STORIES_QUERY`. In the result object we need a data property, and inside of data, we'll have stories where the value is an array of mocked stories. I'll have one here with an id of just 1. The name will be story 1. The image can just be `story1.jpg`, and the description is just test description. Now we need to just pass this mocks array to the `MockedProvider`, and remember to pass the `addTypename` property of value of false. And as you can see, the test is passing. Let's go ahead and make it fail so we can see what the DOM looks like by misspelling the word loading. In the console output here, we do see the DOM is actually showing the loading text. So the component in our test is rendering correctly. Okay, so now that we have a test that is showing the initial loading state of the component, we should get a test to show how the component looks after it actually fetches the data, after loading has finished. This test will be called, should display a story after a successful query. I am going to just copy the mocks object and the render call from the first test and paste it into this new test. So how do we get past the loading message? How can we let our test know that the request is complete? Well, React Testing Library has a utility called `waitFor`. This method is used for testing asynchronous code, allowing you to wait for any period of time for your expectation to pass. So what's going to happen is, if I use `waitFor` within this test, I'm going to have an expectation that checks to see the title of the story is displayed on the page. `waitFor` will wait until that expectation is true, which should happen after the mocked request we make completes. In the `waitFor` callback, I'm going to test if the title is in the document. So the expect argument will use `screen.getByText` again, searching for `story1`, and I want to see that in the document. I also want to test the story description is displayed in the



document. Since `waitFor` returns a promise, we will want to make this test callback asynchronous, and await the `waitFor` call. Let's check the test runner to make sure all tests are passing. All green. We should also check to see that the image has the correct Alt tag content. It should be the title of the story, which in this case would be `story1`. I'll check the Alt tag by using `screen.getByAltText`, and verify that it's showing in the document. If I show the test results again, everything should still be passing.

Testing Error States

Now, what if there is an error? In the stories component, we want to show the text error on the screen when an error happens during the request, so we definitely want to have a test for that. Create a new test and name it `should display an error message on error`. Make the callback `async` again, as we'll be using `waitFor`. Let's create the `mocks` array again and add an object to hold the requests, which will be a query for `STORIES_QUERY`, and this time, instead of a result object, we'll add an `error` property. This is how we trigger an error during this request. The value of the `error` property will be `new Error`. A quick side note, the error that we are handling here is more like a network error. So if the request you're trying to make returns a 404 or some other type of error, this is how we would mock that in our test. Later, we will go over how to mock a GraphQL error or an Apollo Server error, which looks a little bit different. To finish up this test, let's copy the render call from the previous test and paste it here. Now we'll, again, call `waitFor`, and in the callback use `screen.getByText` to look for the error text to exist in the document. Checking the test results, it looks like this test is passing, so to quickly show that we are actually finding the error text, we can force this test to fail by misspelling the word `error` in the regular expression. Now we see the test fail, and we see that error is actually being displayed in the DOM, so the component is working as intended. Now like I was saying before, the error that we are using in this test is a general request error. It's important for our component to handle these errors so that the user can know what's going on. But there's also GraphQL-specific errors that we can also handle to give an even better user experience. For example, our request could have a GraphQL error and still return partial records. In that case, we can still show some data to the user instead of only displaying an error message. Head back over to the ApolloServer code and code sandbox. We're going to simulate an error coming from the ApolloServer by adding an extra field to our story type, and we are going to create a resolver that will throw an error when that extra field is accessed. So here in the story definition, let's add a field called `extra` and make it type `String`. Then in the `resolvers` object, we need to add a `story` resolver as the top key, add an `extra` function right below that, and we want this to return an error, that is just `return new Error('this is not a real field')`. Now this is just going to trigger an error whenever we try to send a query looking for the `extra` field. We can test this out over in the browser here by doing a query with the `extra` field. Here are the errors



that are coming back. There's our message, this is not a real field, but we still have data being returned. Back here in the stories component, let's add the extra field to the `STORIES_QUERY`, and we are going to actually update the error policy in our `useQuery` hook. If you remember from earlier, we did talk about the different options for the error policy, and the one recommended option is the `all` policy. This policy allows us to get back GraphQL errors, along with any data that does successfully get returned. So go ahead and add an object as the second argument to `useQuery` and pass the `errorPolicy` of `all`. We are also returning the error text if there is an error. Well, we don't want to do that anymore since now we want to, one, show an error and also show any successful data. So we are going to move this error line down into the render method so that we can render the error along with the stories. We only want to render the error text if error is not undefined. And as you can see here, there is our error text along with the four stories because the error that we are triggering in the Apollo Server does not affect the four stories being returned. Now let's go back to our tests and update our previous tests by adding the extra field since those would now be broken since we updated the query in the stories component. Now let's add a new test, and we'll call this should display an error message and the successful stories. This test will also be `async` in our `mocks` array. We will have the request object with the same `STORIES_QUERY` query. And now what's going to be different is the result object. We'll want to add an `errors` object, which will be an array with a new GraphQL error that we're going to pass. This error message will just be error with the apollo server. We also want to grab the data objects and return that in the result as well. This is essentially what we are doing in the stories component where we will get an error back, but we will also receive the data. So just copy the render method from a previous test. We are going to await the `waitFor` again, and we're going to expect to see that error message displaying in the DOM. The error message was error loading all of the data, and we also want to make sure that we can see the title of story 1 showing in the document as well. Now we are getting an error here where the test is not passing. Since we updated that error policy on the `useQuery` hook, we do need to make a change here to the `MockedProvider`. We need to let the `MockedProvider` component know that we are using the `all` error policy. To do that, we need to use the `defaultOptions` property on the `MockedProvider`. The object that we want to pass to `defaultOptions` will be the `watchQuery` object. This just means we are looking for any queries run using the `useQuery` hook. We can pass the error policy to the `watchQuery` object and set that to `all`. Now if we check our test results, we should be getting all greens.

Implementing a Mutation

In this clip, I'm going to demo how to use the `useMutation` hook. This is going to be necessary as we add some functionality to our demo app. We want to have the ability to edit the story name. So in the Apollo client, we'll need the



useMutation hook, and on the Apollo server, we will start using a mutation resolver. In this clip, we will build all of this functionality, and then later on, we will start testing it. So back here in the stories component, let's start updating our div here so that we can actually start editing the story name. First, we can remove this anchor tag, and let's go ahead and add a Edit button right inside of the h3 here next to the name. We'll go ahead and use some Bootstrap classes to add some styling. And right beneath here, we want to show a little form for editing the name, so we'll have an input, and we'll have a Save button. Quick side note, the form-group class, the btn class, and the btn-primary or btn-secondary class, those are all classes that come with Bootstrap. If you want to see more documentation and all the classes that come with Bootstrap, just visit getbootstrap.com, and you can look at all the different components. Now we're going to need a function to handle the input field changes. We're going to call that `handleNameChange`. And since we have multiple stories here, we can't just have a single name that's going to change. We need to make sure we have an object that holds the possible values for all the names. So we're going to use the `useState` hook to create and edit inputs object. This object will have a ID as a key, and the value will be the value of the input field. And so every time this input value changes, we call `setEditInputs` with the correct values. Now we add the `onChange` handler to the input field. We pass in the events and the `story.id`. Now we should be good. Next, we need to update the value of the input field by passing in the `editInputs` with the correct ID. Now we want to only show this form if this particular story is in edit state, So we're going to want to create another state property called `editState`. This will be another object where the ID will be the ID of the story. And if that specific story ID is in edit state, we'll show the form. We'll add the `onClick` event to the button. We'll call this function `showEditField`, and we'll pass in the story. The `showEditField` function we'll just call `setEditState`, and we'll set the ID to the `story.id`. We'll also spread in the original value of `editState` just in case we have some future feature that needs it. Now we want to add an event handler to the Save button, and we're going to want to call this `handleEdit`. That'll also take in the `story.id`. Now in the `handleEdit` function, we're going to want to start using the `useMutation` hook. So first, we need to import that, and then we're going to create the query for the mutation. Let's call this query `EDIT_STORY_NAME`, and we need to use `gql` again. So this is going to be called mutation `EditStoryName`. And we don't know what it actually looks like yet because we haven't created it. When you call `useMutation`, it returns a tuple where the first property is the actual mutation function, and the second property is an object that is similar to what `useQuery` returns, which is the loading error and data properties. Right now we don't need that object yet, so we are just going to worry about the mutation. Head back over to the ApolloServer code sandbox, and we need to add the mutation to our type definitions. So inside of the mutation type, we want to add an `editStoryName` function. This will take an `id` of type `ID!` and a `name` of type `String!`, and we're going to return a story. Now we need to add a resolver for the mutation. So just like the query, we just need a mutation key. Inside that objects, we'll have the `editStoryName` function, and



we're finally going to use the `args` argument. This is what holds the ID and the name of the story. Since all of our stories are not in a database, but just a flat file, we are going to use `readFileSync` to read the JSON and assign the value to `rawStories`. Now we're going to just call `JSON.parse` on the `rawStories` to get the actual JSON object. We'll assign that to `stories`. Now we're going to want to loop over the stories, and if the `story.id` matches the `args.id`, we're going to update that name. Now we want to use `fs.writeFileSync` to update the `stories.json` file with our updated stories objects. So just `JSON.stringify`, our `res` variable here, and we want to return the story that we updated. So we'll just run a filter on the `stories` object and grab the one that we just updated. Now I got a little typo here. That should be `story.id`. Now over here in the browser window, we can test out our new mutation. Go ahead and add mutation and then `editStoryName`. We're going to set the id to 1. We're going to make the name New Story Name, and we want to tell it to return the id and the name properties. And if we run that mutation, oops, it looks like I forgot to save the index file. Let me go ahead and save that. Now, we should see over on the right a data property with our mutation name there and our updated story. And if I go and open up the `stories.json` file, we'll see that the updated story name is here in place. Now back over in our stories component, we can fill out our `EditStoryName` mutation. So we're going to specify the id argument as a type ID! and the name argument as a type String!, and then we're going to basically type in the same thing we have in the code sandbox window to actually run the mutation, so `editStoryName`, `id`, and `name`, and we're going to return the id and name of the updated story. So now back in our `handleEdit` function, we're going to call `editStory`, and we're going to pass in an object with a `variables` property so that we can specify what id of the story we want to edit and the name that we want it to be. Now if we go back to the browser, we can go ahead and test our new code. If I click Edit, I should see the form. I can type in a new name here, click Save, and I should see the name updating in real time. Yep, there we go. Looks like it's working. We can even update a second one and change Kid Climbers to Wall Climbing.

Testing Mutations in React

Let's get started writing a test for our new `useMutation` use case. We'll name this test should be able to edit the name of a story. This test will also be asynchronous. Let's just go up and copy the render method from a previous test, and we can get rid of the `defaultOptions` object here, as we won't need it yet. We need to define our mocks array. The first request will be the `editStoryName` request, so let's go ahead and import that query from the `Stories` component. Just paste that here. We'll add the query. Now we need to make sure we add the two variables, the id of the story and the name of the story. The id is the story we want to update, and the name is the new name that we want to set the story to be. For the results, we need a data object and an `editStoryName` object, which should be the story returned from the Apollo Server. Since the



stories query still runs when the component loads, we need to make sure we have a mock for that as well, so let's just copy that from a previous test. Now we want to call `waitFor`. The first expectation is that we still see story 1 showing in the document. Next we're going to use the `userEvent` helper to perform some user actions. We want to be able to click on the edit button, so we'll need to import the `userEvent` method from `@testing-library/user-event`. And we're using `click`, and we just pass in the element that we need to click. We actually are using these attributes called `data-testid`. This is a common practice when writing tests for a React application. Instead of using something like a class or an id, a `data-testid` is something that has no other functionality attached to it. A class name could change or an id could change, which would mean you would need to update your test as well. Testing library has a `getByTestId` method, which makes using test IDs very easy. Next we'll want to type into the input field what we want the new name to be. We also have a `data-testid` on the input now called `input 1`, and we want to type in a new name. There's the `data-testid` on the input, and now we need to click the save button. So instead of `data-testid` for the button, we're going to use the preferred `getByRole` and pass in `button`. And we can pass an object where the name of this button should be the text that displays on the button, which is `save`. Again, I'm going to use a regular expression here. Now to make sure the `useMutation` is actually called, we're going to make a variable, and we're going to set it to `false`. When the mutation executes, we're going to set it to `true`. Now, to test this, we can actually change our `result` object here. It can also be a function. And for this test to pass, that means this `result` function would execute. So we can still return our `data` object, but we can also do other logic like set this `mutationCalled` variable to `true`. Then we can expect this to be `true` in our test to make sure everything's actually working. So I'm going to call `mutationCalled = true` here. Then we're going to expect that that variable is `true`. So a quick recap of this test, we are loading the component, we're making sure that story 1 is displaying, then we are performing a couple of user interactions. We're going to click the edit button, we're going to type in the new string, a new name, then we're going to click the save button. Now we're going to expect that the mutation was actually called. And if we open up the test results, there we go. We have a passing test. Now we still want to make sure we have a test for an error. This will be similar to the `useQuery` test, as they both return the same type of error objects and are handled in the same way. So let's name this one `should display an error message when a mutation fails`. We can go ahead and copy the `mocks` object from the previous test since that doesn't really change. The biggest change we're going to put here is, instead of that `result` object, we're going to pass the `error` object, just like we did in the `STORIES_QUERY` test. The `error` object will just return an error message. We can go ahead and copy the `render` method and the first expectation from our previous test because those still would happen when the component first loads. We can also copy the `userEvents` from our last test, since these are the same actions that we want to run for this test. The expectation that we want to verify is that we see `error editing story name` in the document. Now if we see the



test results, oh, we're failing. Oh, it looks like I forgot to grab the request object for the `STORIES_QUERY`. Let me just copy that and put that into our mocks array. Actually, we also need to update the component to put this error message into the code. We haven't done that yet. So head over to the Stories component. We need to add the error that's going to hold the error objects returned by the `useMutation` hook. We're going to name this `editErrors` so it doesn't conflict with the `errors` object from the `useQuery` method. Now down here in the HTML, we're just going to show the error message if `editErrors` is defined. Let's go back and copy the error message, error editing story name. Oh, and the last thing that we need is, we need to add an `onError` event handler to the `useMutation` hook. We do this by passing in an object as the second parameter to `useMutation`, which is for any options for the hook. For the `errors` property to be defined, we need to have some type of error handler there, even if the handler isn't actually doing anything. I'm not exactly sure why it works this way, but hopefully in a new update to the library, maybe they remove this requirement. Now if we check our test results, we're all passing.

Unit Testing an Apollo Server

Unit Testing Resolvers

In this module we are going to focus on writing tests for the Apollo Server. We're going to write unit tests for the different query resolvers we have in the application. We're also going to write tests for the mutation resolver that we wrote earlier for editing a story name. We're going to look at implementing mocks in the server to serve up mock data for situations when we don't have a database hooked up yet. Using mocks can allow a front-end developer to continue working while the back end is still being worked on. We're also going to look at how we can build a schema using a tool called introspection. Introspection is important when the schema is written in a different language other than JavaScript. You can use introspection to generate a schema into JavaScript so it's easier to use. So let's get started back in the Apollo Server CodeSandbox. For running tests for the Apollo Server we are still going to use `jest` and `babel` just like we're using in the React application. That's one of the great things of using `jest` to run your tests, you can test your Node code or you can test your React components. Make sure to pull in `babel` so we can use imports in our tests. Also grab `babel-jest` and, of course, the `jest` library. Before we write tests for our query resolvers we might want to clean this index file up a bit. So let's go ahead and start moving the resolvers into their own files. This will make it easier to test as we don't have to have the index file run every time we want to import a function from there. So go ahead and create a folder called `resolvers`. And we're going to put a `Query.js` file here, which will hold the resolvers for queries. Just copy and paste the `stories` function over. Assign that to `stories`. And we're going to export the



stories function at the bottom. And remember to import the fs function here at the top. Back in the index.js file we need to require our queries into the resolvers object. Now we can create our resolvers.test file. Let's go ahead and create a describe block. Name this Resolvers. We need to import the stories resolver from our new file, Query.js. Now this first test will be making sure that the stories query returns a certain number of stories. So we can create a values variable and assign the value the execution of stories. Now this, as is, will actually read the stories.json file so we know that we can expect the values array to have a length of 4. Let's make sure we have a test script in our package.json. It looks like we need to add test and assign it jest. we're getting an error for importing. Oh, we did not add a babel config to support using imports. So let's do that now. Create a file called babel.config.js. And we'll want to export a presets array and use the babel/preset-env plugin, setting the target to current. This will allow us to use the import syntax in our tests. Now if I run the test again with yarn test, there we go. Now before we start mocking out the read file functionality and the write file functionality, let's just write a test for our mutation. Again, we want to move the mutation resolver function into its own file. So create a file called Mutation in the resolvers folder. Cut and paste the editStoryName function into the new file. Assign it to the editStoryName variable. And we'll export it just like we did with the query resolver. Again, make sure to import the fs library. And back in index.js, we'll require the Mutation.js file. So this test will be called editStoryName. It should update the name of a story. So for this test we're going to create a mockStories object. This way we aren't actually updating the stories.json contents. The first object will just be a story with the id of 1, story name of story 1, a simple image name, image1.png, and description 1. Let's just copy that and update the number 1 to 2. Now we're going to want to mock the fs library. So, first we need to require that into our file. And we'll use the jest.mock function to mock all function calls to the fs library. I'll put a TODO here to remember to update this test to use the mock fs. We can use the function mockReturnValue, which comes with jest, to tell it when we call fs.readFileSync to return the mockStories. Now we can call editStoryName. We'll want to pass undefined as the first argument, which is the parent object. The second argument will be the updated story object. So it will have the name, which will be the new name we want to assign to the story, and the id of the story we want to update. Now we can expect that the fs.writeFileSync function was called. First we need to import the editStoryName function from the mutation resolver. Now if I run this test I'll add the flag watchAll so that I don't have to keep typing yarn test. We should get another passing test. Oh, I didn't save the resolvers test file. Let me save that. Oh there we go. So we are getting a failing test because it looks like since we are mocking the fs library now that first test shouldn't pass because it's not actually going to read the stories.json via the readFileSync function. So let's just copy these mock stories and put them into the first test as well. We'll update the expectation to just be 2, and there we go.



Apollo Server Mocks

In this clip, I wanted to go over setting up mocks for your Apollo server. Let's take this example. Let's say you are working on the client side of your application, you're building out your React components, and you really want to see some real data coming through. The problem is, the server is not ready to start sending real data. The actual Apollo server is running, and the service is running, but there is no real data coming out of a database that can be used in the client side. To get around this issue and allow the client side to continue being developed, we can just serve mocks up through the Apollo server. Setting up the Apollo server to return mocks is incredibly easy. All we need to do is set the `mocks` attribute in the `ApolloServer` config to `true`. So back in the code sandbox in your `index.js` file, add `mocks: true` to your `ApolloServer` config. Now if we go over to this browser window here, we can start running queries against our Apollo server, which should now be returning mocks. So let's run a query that returns stories, and we want the `id`, `name`, `description`, and `image` properties. We can see here in the results that we now have mock data coming back. The IDs are strings. They look like UUIDs, and all of the string properties, `name`, `description`, and `image` are all returning `Hello World`. These are default mocks generated based on the type of property that we are looking for. So if we look back at the type definitions for the type `story`, the type `id` is giving us a UUID, and all of the type strings will give back `Hello World`. This is obviously not the best, most customized approach since every string looks the same, but it is a starting point. It does allow you to return some data to the client side so that you can continue building out the application. Now if you notice, we only have two stories being returned. This is due to the fact that the default setting for mocks is just going to be 2. Well, if we want to actually customize the mocks that are being returned, then we don't want to just pass the Boolean `true` to the mock's config property. We can instead pass a `mocks` object that will tell the Apollo server to return a more specific set of mocks. For example, we can say for all queries against the `stories` object, we want to return a specific set of stories or for all requests for the `id` property, we want to always return a number between 1 and 10. Let's create a `mocks` object and customize the `stories` query. First, we want to import the `MockList` function from the `ApolloServer` library. Now the `mocks` object is going to look similar to the `resolvers` object. So first, we'll have a `query` key, and that will be a function that returns an object for all of the types that you want to mock. In our case, we are going to mock the `stories` type. So the `stories` property will be a function that returns a `MockList` object. We want to instantiate a new `MockList`, and we're going to pass in the value 4. What this means is this will generate an array of four story objects, and those will be returned every time we do a query for stories. To see this in action, let's run this query again over in the browser and still returning to, oh, I did not update the `mocks` object in the `ApolloServer` config, so let's just remove the Boolean here. And when I run this query again, we are getting back four stories now. Each story is still being mocked



with the UUID for the id and the Hello World string for each string property. Let's not stop there, though. Let's add a little bit more customizability to these mocks. Let's go ahead and add the library faker so that we can get some real-world looking data to generate when we use the MockList. So we'll just head over to the dependencies, click Add dependency, and type in faker. We can install the latest version. Please check out the faker documentation by visiting the link below. It'll have all of the different types of data that you can have generated that will look like it's coming from a real database. I'll be using a few different properties, but if you want to see the full scope of what can be generated, take a look at the documentation. Now just require the faker library at the top of the page. Now back down in the mocks objects, we're going to define a mock for the story type. The first property we're going to mock is the name. And for this property, we're going to use `faker.company.catchPhrase`. We know that our story names are usually two or three words, and `catchPhrase` is going to return something like that. Now for the story description, we're going to use `faker.lorem.sentence`. This will return a Lorem ipsum-type sentence, and for image, we're going to use the faker image property. The image property will return a URL to an image. The size will be based on the dimensions we pass here. In this case, we're going to pass a width of 200 and a height of 400. The Boolean I'm passing as a third argument is the randomized property, so I'm going to put `true` to make sure it's random. We are also using the `nature` function to get a nice nature-related image back. Now let's run that same query again, and we can see our updated information. Now the names are all different. They're not just Hello World anymore. The descriptions look like actual descriptions of products, and our image is coming from `lorempixel`, and those are actual images. So you could see our app is getting some real-looking data coming back in our mocks. Now we're still just returning four stories, but what if we want to change that number? Maybe you don't want to return a single number on every request, but maybe you want to see a different number of stories return on each request, maybe a range between two different numbers so that you can see what if I only have one story coming back or what if I only have 10 stories coming back? To make this possible with MockList, we can, instead of passing an integer, pass an array with two different values. The first value is the low end. The second value is the higher end of the range you want, so I'm going to pass 6 and 12. This means I want anywhere between 6 stories and 12 stories to return on each request. So if I run that query again, the first time I got 12 different stories coming back. I can run this again, and this time I have 10 stories coming back. So anywhere between that first number and that second number is what you would expect.

Nested Mocklist Demo

In our previous example, I showed you how to use MockList to generate data in two different ways. We either generated a set amount of data or we generated



data within a set range, so anywhere between 1 and 5 items or 10 and 20 items. Now we're going to do something a little bit more complex. I'm going to show how to generate nested lists. For example, we're going to add reviews to each story, and each review will be a list of reviews for that specific story. So we'll be generating a MockList of stories that also has a MockList of reviews. Let's define the reviews property on the story type in the type definitions. This type will be a nested array of ratings. We'll define the type ratings in a second. The syntax for a nested array looks like this, it's basically just an array inside of an array. For the type rating, we're going to have three different properties. We want to have the lowestRating, which will be an Int. We want to have the highestRating, another Int, and we want the date, which will be of type string. You can imagine this is more of a culmination of ratings for a specific date. So let's say on January 1st, the lowestRating received was a 1, and the highestRating received was a 4. So what we want is to have reviews return an array of multiple arrays of ratings. So we're going to have three reviews come back, and within each review, we will have a list between 1 and 3, which will be the ratings. So I'm going to add as the second argument to MockList a function that returns another call to MockList. This will make sure we're getting anywhere between 1 and 3 ratings back. Now we need to mock out the ratings properties so that we can get some realistic data looking when we run this query. We'll continue using faker for this. We can use the faker.date.past method to get a date sometime in the past. For lowestRating and highestRating, we want to make sure that the number is only going to be between 1 and 5, so we can use faker to give me back a random number between a set list. So we'll use the arrayElement method, which takes an array where each value in the array is a possible value that should be returned. We're going to just use the numbers 1 through 5. We can just copy this line for highestRating, as it'll be the exact same thing. Now we can add reviews to our query over in the browser window, and let's make sure we get back the date, the lowestRating, and the highestRating. And if I run this query, you should see the reviews coming back, an array of arrays of ratings. So there's our date, the lowestRating and our highestRating, and those numbers are going to be random between 1 and 5.

Introspection

As we've talked about before, introspection is a method to get the schema of a given GraphQL server by running a query against that server. Similar to a normal query, we are going to run a query that will return how a schema is defined. This is something that you would do if you, for example, didn't have access to the source code of a GraphQL server, but you want to know how to structure your queries based on the type definitions. This is very common when you're working in an environment where the back end is written in a different language, like Java or Python, yet the front end is using JavaScript and the developer writing the queries needs to know what the schema looks like. This isn't specifically an



Apollo function, it's more a GraphQL function. In this clip we are going to write a script that will make an introspection request to our CodeSandbox Apollo Server to get the schema definition. And instead of using the type definitions that we've defined in our index.js file, we're going to use the schema that we pulled back from that introspection query. This way you'll get a sense of how to actually use introspection. Now as I said, this will be a script, a Node script to be exact. So let's get started by creating a new JavaScript file. We're going to call it getSchema.js. First, we're going to import the getIntropectionQuery method, and that comes from the GraphQL library. And if I console.log what calling this method does, you'll see the query that we're going to send to our Apollo Server. As you see, there are a lot of different options here, but the important one is the `_schema` there where we're getting the `queryType`, the `mutationType`, the `subscriptionType`. Now we don't have any subscriptions in our schema, but the `getIntropectionQuery` basically looks for all possibilities. We won't get too far into the details of what's inside of this query as we aren't going to actually edit it or add anything to it. We just wanted to see what is the query that gets generated by the `getIntropectionQuery` method. Now to make the request to our server we need to first add the `node-fetch` package so that we can use the `fetch` API inside of our Node script. Once you have that, go ahead and create a `fetch` variable and require the `node-fetch` library. Now we can define our `getSchema` function. It's going to be asynchronous. Then we're going to want to call `await fetch`. And go ahead and copy the URL from the codesandbox browser. The second argument to `fetch` for a POST request will be setting the method to POST And we'll add a `Content-Type` header for `application/json`. We'll add an `Accept` header, also for `application/json`. And now we want the body. Now the body will be the actual query returned from calling `getIntropectionQuery`. And we want to stringify that value. Create a variable called `json`, and we'll assign to it the value we get back from `response.json` resulting. So `await that`. Actually, let's not call this `json` Let's call it `result`. Now let's just `console.log` out that result. And let's wrap this whole thing in a `try/catch` block just so we can make sure we handle any errors that might occur. If there is an error, let's `console.log` it in the Node process. Now at the bottom let's call `getSchema` to execute this function. Go over to the terminal and run `node getSchema.js`. Okay, it looks like we have an error. Let's see what we did wrong. Oh, it's the body. We need to pass an object with a query. So let's go ahead and add the `query` key. And we'll just wrap the `getIntropectionQuery` call in backticks so it renders as a string. Okay, run that again in the terminal, and there we go. We see our response. Now we know that our request is working, so we need to do something with the response. We need to save that query to a file so that we can use it in our actual Apollo Server. So let's require the `fs` library. And let's go ahead and use `fs.writeFileSync`. And we will write these results to a file called `introspection-query.json`, we'll need to stringify the results to add them to the file. Now if we run this script again we should have a generated file. And yep, there it is. There are a lot of objects here and a lot that goes into this query, but again we aren't going to touch this file or edit this. It will just work right out of the



box. Now we need to head back over to our `index.js` file, and we need to build this schema to use in the Apollo Server config. There is a handy function called `buildClientSchema` that we can pull from the GraphQL library. Let's also pull in the schema from `introspection-query.json`. We'll call it `schemaResult`. Now we can create a variable called `schema`, and we're going to call `buildClientSchema`, passing in the `schemaResult.data`. The `data` property holds the actual introspection query. Now down here in the Apollo Server config, replace the `typeDefs` variable with the `schema` variable, and that's really all we have to do to use the generated schema. We can test it by just coming over to the browser window and running a `stories` query again. Let's pull back the `id`, the `name`, and the `description`. And as you can see, we are still getting back all of the data in the correct formats. We still have the `ids` looking like UUIDs. Those are all based off of our mock data. We still have our names as strings, our descriptions as strings. We didn't even have to use our `typeDefs`. The introspection query figured out what the schema looks like so that we can still access the data. Let's not forget about `image`. And there we still see the `lorempixel.com` images coming back. So again, if you don't have access to the type definitions or the schema on the server side because it's in a different language or for some reason you just can't see the source code, using the introspection query is going to really help you out.

Integration Testing

Integration Testing

In this module, we will start writing integration tests. What this means is we will be executing queries against our Apollo Server using our actual Apollo Server config, but we will use a test client so that we are actually making HTTP requests and changing actual databases. The `createTestClient` function from the Apollo Server library allows us to do just that. We will also be looking at some Jest snapshot tests, which means we can run a query against our Apollo Server, get a snapshot of what the results should look like, and then, in all future runs of the same test, we will make sure that those snapshots always match. If for some reason that snapshot did not match, that means maybe we change something in our source code that we didn't want to change. Snapshot tests just kind of make sure that we aren't doing something unintentionally. The first thing we'll want to do is create a new file called `integration.test.js`. Next, we'll need to install a dependency; it's called `apollo-server-testing`. It's a library that gives us tools to create a testing client that we can use instead of actually accessing a real Apollo client. So go ahead and install that. Now over in `integration.test`, we're going to require the `createTestClient` function, and we're going to use this to create an Apollo Server client that's only going to be used for tests. This will allow us to actually write integration tests that don't actually touch real data or go out and



make any HTTP requests. Since we're still using Jest, let's create that describe block, and call that Integration Test -Query, since we'll be testing queries here. In our first test, we'll want to call this, should return a set of stories. Now we can actually create an Apollo Server instance. We're going to call that server, and we need to pull in the ApolloServer function from the apollo-server library. Now, just like in the index file with our real Apollo Server client, we have to specify the type definitions and the resolvers. Now for the purpose of this test, let's just copy and paste those from the index file into our test file. Let's make sure we include the gql function. Okay, now we can fill out the config for our Apollo Server. I'm going to go ahead and remove this story resolver, since we don't need to use this anymore. Let's also bring over the mocks from the index file as well. And remember to import the MockList function, and also pull in the faker library. Now we can call createTestClient and pass in our server, which is an ApolloServer instance. And what we're going to get back is a query method. We're going to use this query method to run queries against our test Apollo Server client. This query function does return a promise so we can await that, and we're going to pass an object as an argument with a query key. Now we can just write a query here, similar to what we use in the browser window over to the right. Let's do a query for stories, and we're going to pull back the id, the name, the description, and the image. Now we're going to want to write an expectation and get the correct amount of stories that should be returned. The response object, just like we have in the browser window to the right, will have a data object, and inside that data object, it should have a stories object. That object should have a list of stories that are being mocked with the MockList function. We want the length of that array to be equal to, now we're using a range, so let's set that to 5, just a static number. So we'll expect this length of stories to be 5. Let's go over to the terminal, and we're going to run this test by running yarn test integration. I am passing integration here after yarn test, because Jest will just use a pattern match to look for any file names that have integration inside of them. And there we go. Test is passing. The integration test is again set up to use a test client. We are pulling in the Apollo Server function from the Apollo Server library, but using the createTestClient function, we are not going to actually hit a real Apollo Server. We can pretty much test every query or mutation against this test client. Next, we'll look at how we can integrate snapshot tests with this same integration test client.

Snapshot Testing

Now that we know how to implement integration testing with our Apollo Server, we can also learn how to write a snapshot test. If you're not familiar with Jest, a snapshot test is a way that you can test the outcome of a function being called against any other future test runs to make sure something unexpected didn't change with the source code. For example, Jest will take the output of a function call, save that output to a file, and then on future test runs, it will just check to see if the new output from that current test run matches what was



saved in the snapshot file. Let's take a look at a quick example here. To run a snapshot test, all we have to do is add an expectation for this result to match snapshot. This is the specific function you're going to want to use, which will, if a snapshot doesn't exist, it will create a snapshot file; if it already exists, it will do a diff of the new snapshot versus what was already saved. Let's run yarn test integration again. This time I'm adding the `-watchAll` flag so we don't have to keep running it manually. It will run every time we change a file. So we see it's passing, and we see a snapshot was written for one test suite. If we come over to the files, we see a new directory called Snapshots, and inside there is an `integration.test.js.snap`. Here in this file, we can see the object that comes back from that query call. Inside of our test. The snapshot includes the name of the test, and the data coming back from the function that we were actually testing. In our case, this was the result object that is returned from our query call. There you can see all of our stories along with the descriptions, the ids, all of the information there. Now, if I make a change to this file by just adding a new line, click Save and rerun the test, we'll see that it failed. Now I know why it's failing in this case. Remember, we are using that faker library to generate new data for the name description image and the id of each story. With the snapshot test, we're expecting each run to be exactly the same, so we probably shouldn't use a generator for the test. Yes, you can see here how descriptions, id, image name, how each of those are different now. So maybe instead of using this mocks object here, we will create a new mocks object called static mocks. This object will not change on every test run. So instead of using `faker.company.catchPhrase` for the name, let's just use the string `Carved Rock`. Same for description, let's set a string to `Carved Rock Outdoor Fitness`, and for the image, we're going to use the string `image.png`. Oh, we've got to fix the spelling here. I think it's also best if we create a new test. I don't want to have too much stuff going on in a single test. Let's call this test, `Should match all snapshots`, and we're going to want to just copy the Apollo Server client creation from the previous test. We want to make sure we make this `async`. And instead of the dynamic mocks object, let's set this to static mocks. Let's move this snapshot expectation into our new test. Okay, we run it, and we should get a new snapshot. Now if I change a line and run this test again, well, we see it's failing. Oh, the id, we did not set the id to a static string. Let's go ahead and go back to our static mocks object and set the id to return as the number 2. Well, now we have to update our snapshots, since now it's comparing to our original snapshot which has the id as a UUID. To make sure we update the snapshots, just add the `-u` flag after yarn test. This is passed to the Jest command, and we'll update all of these snapshots. Alright, so that created a new snapshot for us. Now let's run our yarn test again, again passing the `watchAll` flag, and we're passing. Now let's change a line, make sure we're passing still. We are still passing. Now what if I change this id from 2 to something different? It should fail, and we see the comparison there. The snapshot was expecting to see the string 2. Let's set that back to 2, and there we go. Now you might be wondering, why do I need the snapshot tests, or do I need snapshot tests? And really, it just depends on your use case and what your team is looking for. The



benefit of snapshot tests is they run really quickly. I think they make you feel comfortable that nothing is really broken with your functions. If you know that a snapshot test has passed, then you know that something big didn't change. It's kind of like a security blanket. Now, the downsides I can see with snapshot tests is when you do want to change your source code, then you're going to have to update your snapshots. It just means it's another step that you need to take to make sure your testing suite is actually being helpful. Some people can see that as being an extra step that you don't really want to take, but try it out and see how you like it.

Course Summary

Well, that brings us to the end of this course. Let's take a look back at what we accomplished throughout this course. We started by going over all of the different Apollo client hooks that we will be using in our application to interact with our Apollo server. We knew that we would be fetching data from our server, which required the `useQuery` hook. And we knew we wanted to add some edit capabilities to each of the stories, which would require the `useMutation` hook. We then used these hooks to add to our Carved Rock Fitness application to make it more dynamic. We wanted to connect it to our Apollo server and read the data coming from a GraphQL database. To make this work, we had to create some new resolvers in our Apollo server. We learned that resolvers are responsible for fetching the data that is defined in our schema and returning it to the client. We also got into mocking out some of our data for those situations where, let's say a database isn't ready to go yet, but we want the client-side to continue development and we want to actually hit an Apollo server. We introduced the `MockList` function for when we have a list of data, or an array of data, that we want generated and returned to the client-side. We also looked at introspection. This is a GraphQL tool to get the schema definition from a GraphQL server where you might not have access to that source code. You can run an introspection query to get back what that schema looks like. And, of course, there was the testing. We learned how to test our React application using the `MockedProvider` component. We tested queries and mutations, including all of the different states, including the error states. On the server-side, we made sure to test the resolvers individually, and we also wrote some integration tests, so that we can get a much more thorough test of our Apollo server. Thank you for watching this course. I really hope you were able to learn something about testing your Apollo application. I wish you the best of luck when you're developing your next Apollo application.